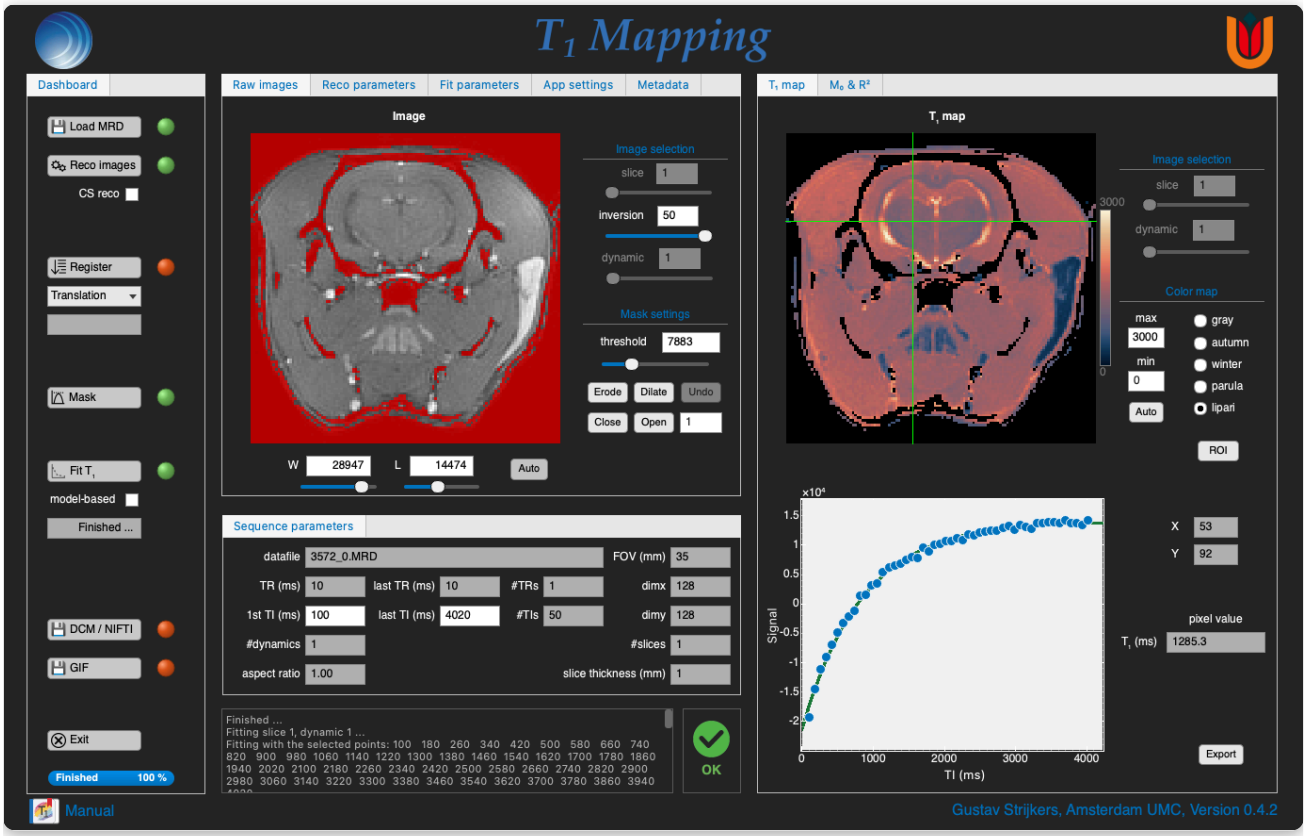


Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Installation
5. Running the software
6. Basic operation
7. Credits

1. Overview

T1mapp is an application with a graphical user interface to reconstruct T_1 maps from inversion recovery (Look-Locker), saturation recovery or variable flip angle experiments acquired with an MR Solutions preclinical MRI system. The images are reconstructed, registered, masked and fitted to T_1, M_0 and R^2 maps, which can be exported as DICOM, NIfTI, BIDS and GIF, each export with a metadata record and a report of how the maps were made, and the values of a region of interest as a CSV file.

T1mapp is a stand-alone application: everything it uses, elastix and dcm2niix included, is inside it, and nothing else has to be installed.

2. Contact

T1mapp and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

3. Disclaimer

This is a beta version. The software has been extensively tested using mouse and rat data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the program will meet your requirements or that the operation of the program will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers.

4. Installation

macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

T1mapp comes as a disk image, `T1mapp-0.4.4-macOS-arm64.dmg`. Nothing else has to be installed: Python, elastix, dcm2niix and everything else the app uses are inside it.

1. Open the disk image and drag **T1mapp** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying T1mapp was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

   Instead of step 2, the same in Terminal, once:

   ```
   xattr -dr com.apple.quarantine /Applications/T1mapp.app
   ```

The first start can take a little longer: macOS checks the app once.

Windows

Requirements: a PC with Windows 10 or 11, 64-bit.

T1mapp comes as an installer, `T1mapp-0.4.4-setup.exe`. Nothing else has to be installed: Python, elastix, dcm2niix and everything else the app uses are inside it.

1. Run the installer. It is not signed, so the first time Windows may stop it ("Windows protected your PC"): click **More info** and then **Run anyway**.
2. The installer puts the app in `%LOCALAPPDATA%\Programs\T1mapp` for the current user, without administrator rights (installing it for all users is an option), with a Start-menu entry and, if you choose, a desktop shortcut. At the end it can start the app.

To remove the app, use **Uninstall T1mapp** in the Start menu, or **Settings > Apps**.

BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (https://mrirecon.github.io/bart/), version 1.0.00 or newer, the model-based T_1 fit is available. Without BART, or with an older version, the model-based fit is switched off and compressed sensing runs on the built-in sigpy engine. The message box says so at start.

Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

   ```
   xcode-select --install
   ```

2. Install MacPorts from https://www.macports.org/.
3. Install the packages:

```
sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils l
ibblocksruntime
```

4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install:

```
export PATH=/opt/local/libexec/gnubin:$PATH
CC=gcc-mp-14 gmake
sudo gmake install
```

   The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

6. Check that it works: `bart version`

Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`
2. Install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/instal
l/HEAD/install.sh)"
```

3. Install the packages BART needs:

```
brew install gcc libpng fftw openblas make llvm libomp
```

4. Download and unpack BART as above (step 4 of MacPorts).
5. Build and install:

```
CC=gcc-15 MACPORTS=0 gmake
sudo gmake install
```

   The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.

6. Check that it works: `bart version`

How T1mapp finds BART on macOS

`sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

Folder	
`/usr/local/bin`	where BART's own `make install` puts it
`/usr/bin`	
`/opt/homebrew/bin`	Homebrew's
`/opt/local/bin`	MacPorts'

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once (Terminal asks for your password):

```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

Installing BART on Windows 11

On Windows, BART runs inside the Windows Subsystem for Linux (WSL); the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run `wsl --install`. This installs WSL 2 and Ubuntu; a restart is needed. The first time you start the Linux prompt you are asked to create a user account. See https://pureinfotech.com/install-wsl-windows-11/ for older systems.
2. Update the Linux distribution:

   ```
   sudo apt-get update
   sudo apt-get dist-upgrade
   ```

3. Install the packages BART needs (gcc 12 or newer):

   ```
   sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenb
   las-dev gfortran
   ```

4. Download, build and install BART:

   ```
   wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
   tar xvfz v1.0.00.tar.gz
   cd bart-1.0.00
   make
   sudo make install
   ```

5. Check from a Windows command prompt: `wsl bart version`

5. Running the software

Open **T1mapp** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows). The T1mapp logo appears while the app loads, and the window follows within seconds. At its start the app looks for elastix and BART, and the message box reports what it found; "Ready" says it is done. Only **Load MRD** and **Exit** are on until a scan is loaded.

A scan given on the command line (an MRD file or its folder, as a program that starts T1mapp with a scan gives it) is loaded at once, as with **Load MRD**; a path the app cannot load gives a warning in the message box.

Newer versions. At its start the app asks GitHub (https://github.com/Moby1971/MRI-apps) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

Updates, in the App settings tab:

- **auto update**, ticked as it comes: new versions of T1mapp arrive by themselves. When one is ready, a window says so with what is new: **Install and Restart** installs it and starts the new T1mapp; **Later** installs it when you quit T1mapp, without asking again. The window does not come while a step runs, but when it has finished. Without an internet connection nothing is said and nothing waits. On Windows, T1mapp installed for all users can be updated only by an administrator; for other users the message box says once where to get the new version to install by hand.
- **Check now** looks for a newer version at once, also with **auto update** off; the message box says what it found. On the Mac, **Check for Updates...** in the T1mapp menu does the same.
- **notify at start**, ticked as it comes, counts only with **auto update** off: when a newer version of T1mapp is available, a window at the start offers it. **Download** gets the installer for this computer, **Skip this version** shows no window for that version again, **Later** asks again at the next start.

Moving to MAPapp. T1mapp continues as MAPapp, one app for the T_1 and T_2 maps, which takes over your settings. Once MAPapp is released, T1mapp says so in a window at its start: "T1mapp continues as MAPapp." **Install MAPapp now** downloads and installs MAPapp, moves T1mapp to the Trash (on Windows: uninstalls T1mapp) and opens MAPapp; **Later** keeps T1mapp as it is and asks again at the next start. When MAPapp is already installed on your computer, the window says so instead and offers **Open MAPapp**, which opens it and quits T1mapp (T1mapp stays installed), or **Later**. The window comes with the check for new versions: with **auto update** on, or with **Check now**; it does not come while a step runs.

The message box says at every start whether T1mapp is up to date.

This manual. The **Manual** button at the bottom of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

Settings and log files

T1mapp keeps its files in a folder of its own, which it makes at its first start:

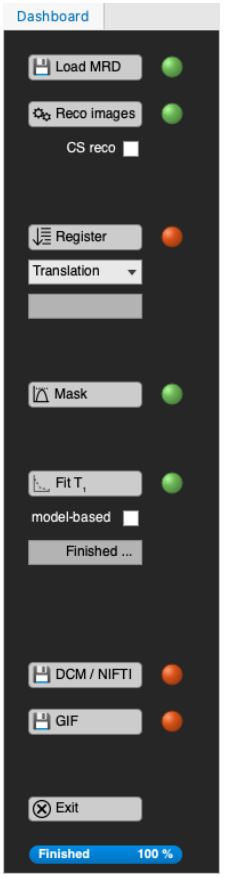
macOS	`~/Library/Application Support/T1mapp/0.4.4p`
Windows	`%APPDATA%\T1mapp\0.4.4p`

Settings. The settings are kept in `t1mappSettings.json`. The file is written with the standard values the first time a version starts, with your choices from the version used before, and can then be edited by hand; changes take effect at the next start. It holds the folders where loading and the exports start (`mrdImportPath`, `defaultExportPath`), the theme, the sounds, the reordering of segmented k-space, the values the reconstruction, registration, fit and display fields start with (the color maps of the three maps among them), and the number of log files kept. A value that cannot be used is replaced by its standard value, and the message box says so; a file that cannot be read at all is set aside as `t1mappSettings-unreadable-<time>.json` and replaced by a new one.

T1mapp keeps your choices of the theme, the sounds, auto update, notify at start, the color maps of the three maps, the registration transform, min R-square, de-noise, de-ring and toggle reorder, and the folders last used for loading and for the exports, for the next start; a new version takes them over.

Log files. Every session writes a log file, `t1mapp-log-<date and time>.txt`, to the same folder. The log keeps every message and the parameters of each import, reconstruction, registration and fit. Logs older than the newest 50 are removed at start.

6. Basic operation



The T1mapp program operates from a single window with 5 panels.

Panel 1: Dashboard. The task buttons and parameters that control the reconstruction process. The tasks run from top to bottom; registration and masking can be left out. A green light shows a task done, yellow a task running, and red one not done or failed. At the bottom of the Dashboard one progress bar shows the progress of every task ("Registration 40 %", "Fit 40 %", "Finished").

Panel 2: Raw images / Reco parameters / Fit parameters / App settings / Metadata. The **Metadata** tab holds the record of the scan: the animal, the session and the contrast agent (step 9).

Panel 3: Sequence parameters. The relevant acquisition parameters; for inversion recovery the first and last TI can be edited, which spaces the TIs again and clears a fit.

Panel 4: T_1 map / M_0 & R^2. During and after fitting, the T_1 maps. The M_0 and R^2 goodness-of-fit maps are displayed in the second tab.

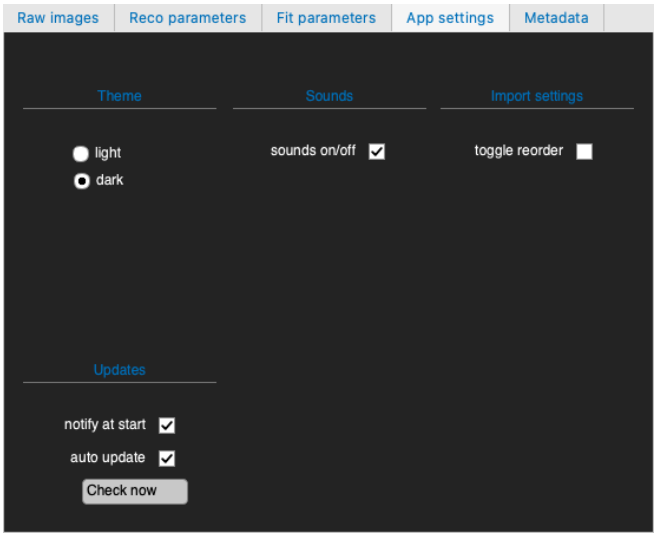
Panel 5: Messages. The program messages, newest on top, and the status: OK, BUSY, WARNING or ERROR.

Step 1: Loading data

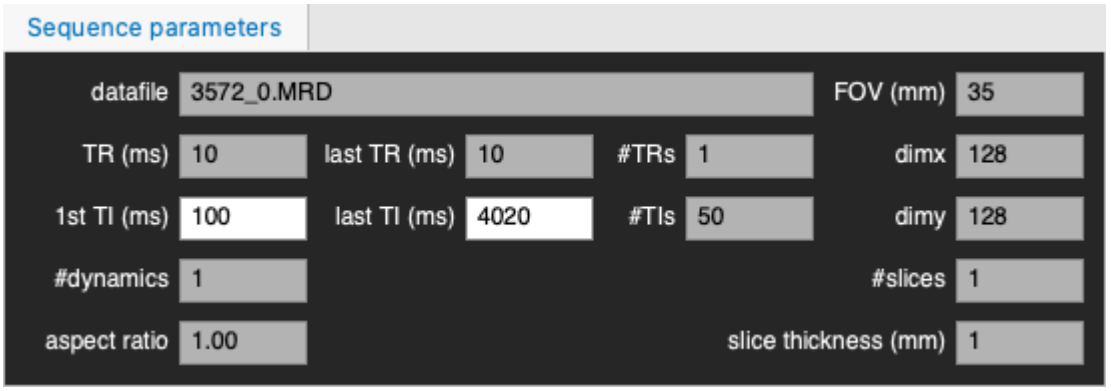
Press **Load MRD** to load the MRD raw data file. On the scanner computer (Windows) the first dialog of a session opens in the newest exam of the scanner's MRD folder, unless the settings name a folder (`mrdImportPath`); elsewhere it opens in the folder of the last file loaded.

T1mapp recognizes three T_1 measurements from the MRD parameters: inversion recovery (Look-Locker FLASH), saturation recovery (multi-TR) and variable flip angle series; other scans are refused, with the reason in the message box, and the scan loaded before stays. Retrospective's and P2ROUD's MRD exports load as well (a variable flip angle CINE from Retrospective, for instance).

If a segmented FLASH scan does not reconstruct properly, tick **toggle reorder** in the App settings tab and load it again.



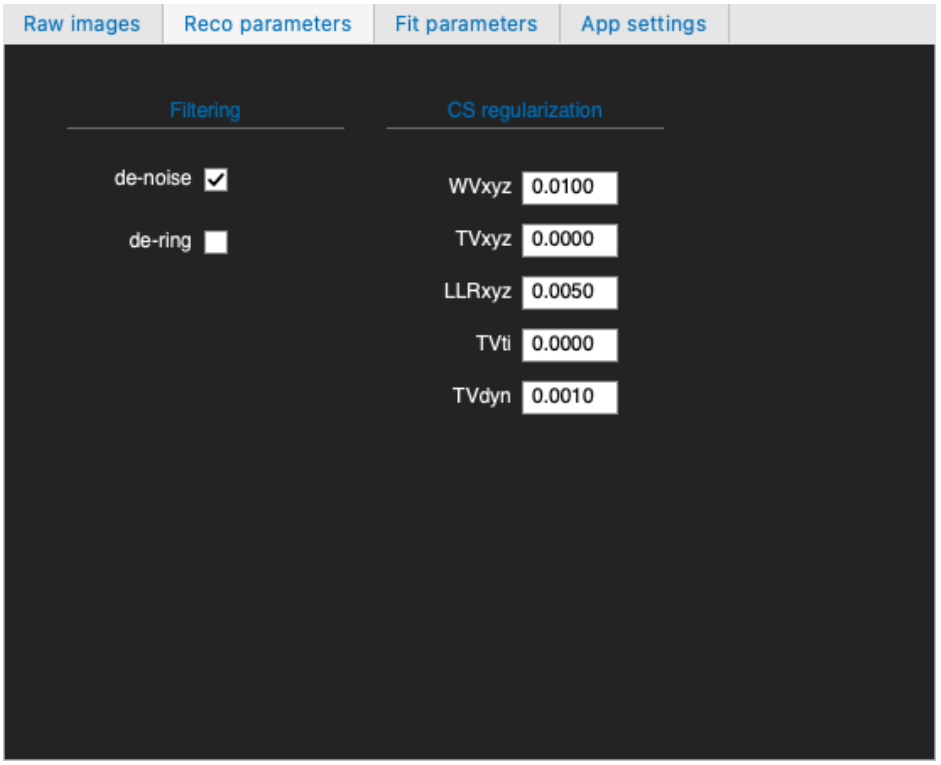
For multi-receiver coil data, select one of the MRD files; the app loads the others. The relevant acquisition parameters are shown in panel 3.



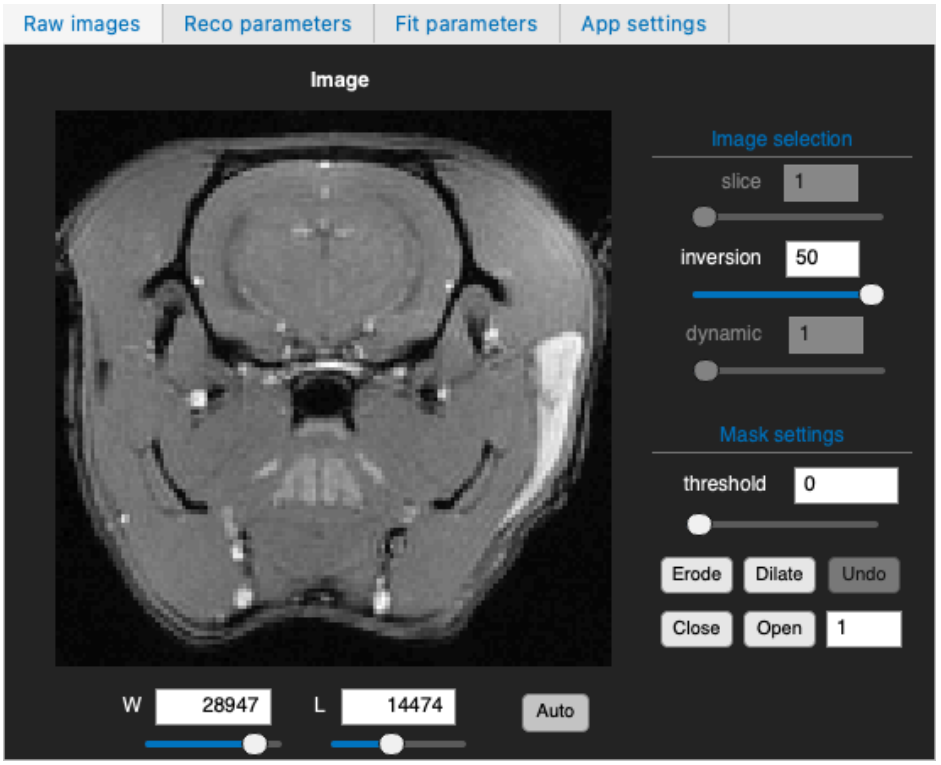
Step 2: Image reconstruction

Loading reconstructs the images already. Press **Reco images** to reconstruct them again, for instance with compressed sensing. While a reconstruction runs, the button reads **Stop**: pressing it stops the reconstruction and leaves the earlier images and the parameters as they were.

The standard reconstruction is a 2D or 3D FFT with PCA de-noising, on by default. When the **CS reco** checkbox is checked, the images are reconstructed by compressed sensing with the weights of the Reco parameters tab: wavelets (WV), total variation (TV) and locally low rank (LLR), in space (xyz) or along the series (TVti, TVtr or TVfr) and the dynamics (TVdyn or TVfa). Compressed sensing runs with BART when it is installed, else with sigpy; both take every setting, and the message box says which engine ran and why. That tab also switches PCA de-noising (**de-noise**) and Gibbs ringing suppression (**de-ring**, off by default), which removes the ripple beside sharp edges.



Images with different inversion times and dynamics can be inspected with the sliders of the Raw images tab (**slice**, **inversion** — or **TR**, **frame** — and **dynamic** — or **flip angle**). The window and level are set with the **W** and **L** fields and sliders, or with **Auto**. The mouse wheel over a slider moves that slider, one step per notch.



Step 3: Image registration

NOTE: this step usually can be omitted.

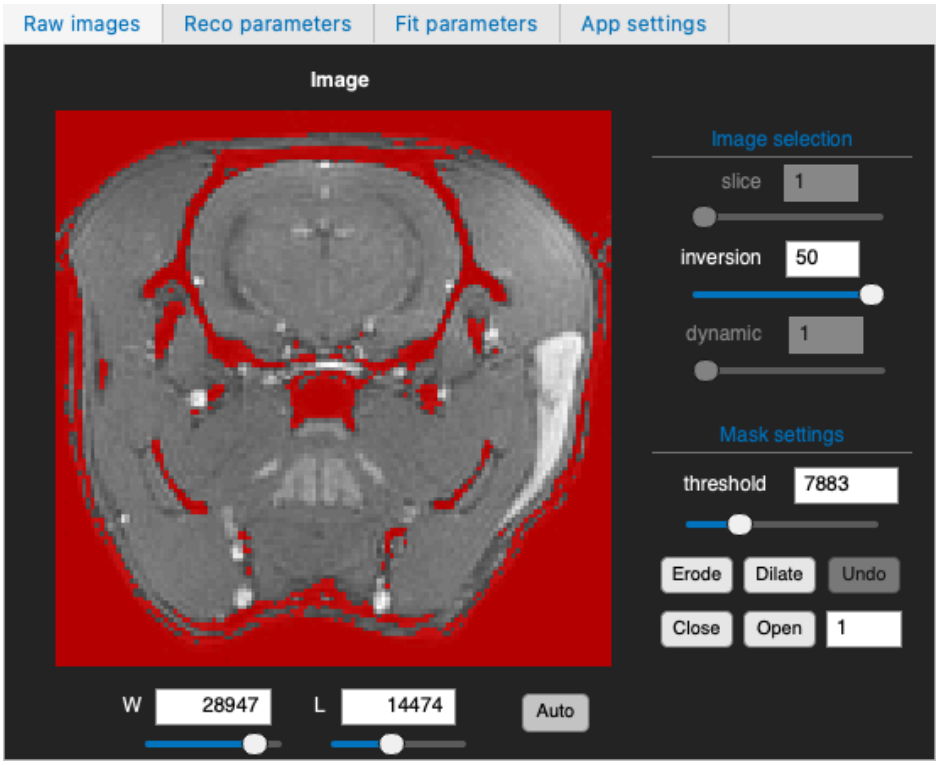
In case the images as a function of the inversion times or dynamics are not perfectly registered, press **Register** to register them as a **Translation**, **Rigid**, **Affine** or **B-Spline** transform (the list below the button). While the registration runs, the button reads **Stop**: pressing it stops the registration and leaves the images as they were. Registering clears the mask and the maps, and rules out the model-based fit.

Step 4: Masking the background

To prevent unnecessary fitting of the background pixels, the images need to be masked first; without a mask every pixel is fitted, the background included.

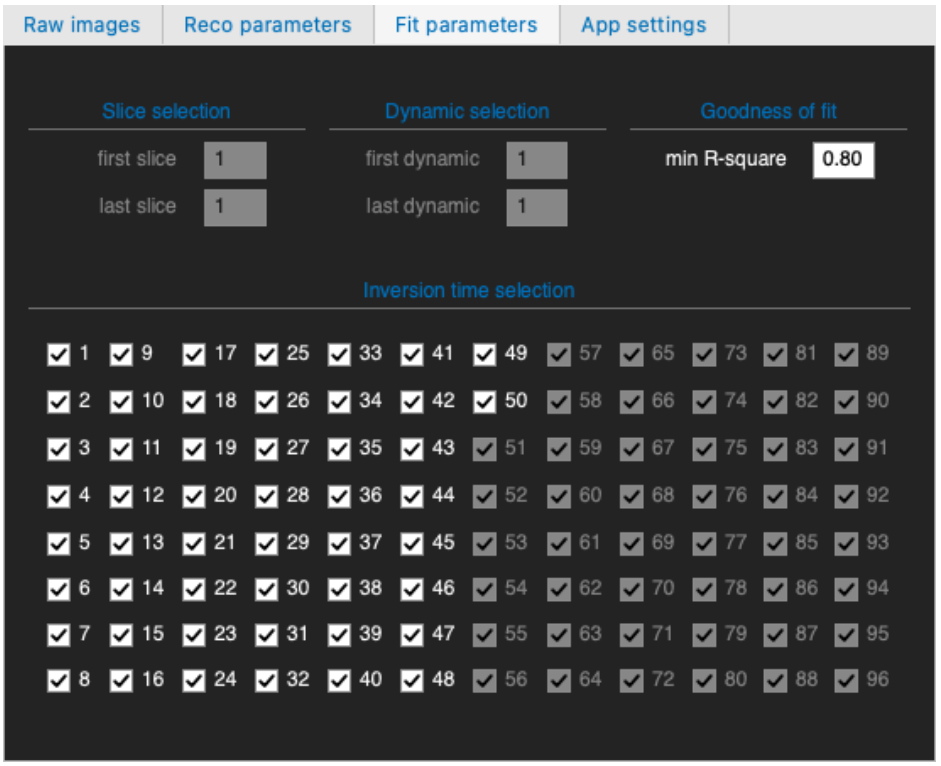
Press **Mask** to mask them with an automatic threshold per slice. The red pixels indicate the regions that will be skipped during T_1 fitting.

In case the automatic threshold is not satisfying, it can be adjusted per slice with the **threshold** field or slider. The mask can be refined by **Erode**, **Dilate**, **Close** and **Open** operations, of the size in pixels in the field beside them; **Undo** takes the last one back. Any change to the mask after a fit, **Mask** included, withdraws the fitted maps: fit again to see and export them.



Step 5: Fitting T_1 maps

The Fit parameters tab holds the fit settings.



The first and last slice and dynamic (for variable flip angle data, frame) that will be fitted can be selected.

min R-square is the lowest R^2 accepted: where the fit of a pixel has a lower R^2 (0.80 by default), or gives a T_1 above 10 s, the T_1 and M_0 values are set to zero.

The bottom part of this tab lists the points of the series in the order of the scan (inversion times, TRs or flip angles): those ticked are fitted, so points can be left out, as long as 3 remain.

Press **Fit T_1** to start the T_1 fitting. The maps fill in slice by slice while the fit runs. Meanwhile the button reads **Stop**: pressing it stops the fit and leaves the maps as they were before it.

For inversion recovery the fit is a least-squares fit of the equation

> $Signal = | A - B \exp(-TI / T_1^*) |$, $T_1 = (B / A - 1)\, T_1^*$

Model-based fitting

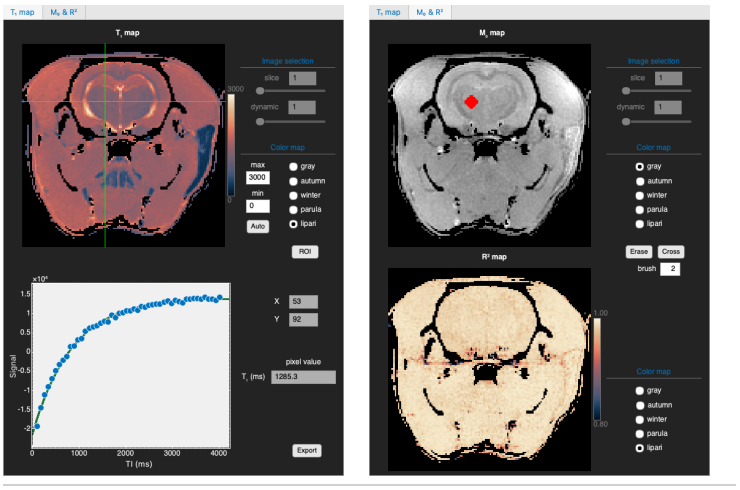
Tick **model-based** to fit a model-based reconstruction with BART: the T_1 map is then fitted to the k-space directly. It is offered for 2D inversion recovery that has not been registered, when BART is available.

Step 6: Fit results

The T_1 maps are shown in the T_1 map tab, on the scale set with **max** and **min**, or with **Auto**. An M_0 map and an R^2 map are shown in the second tab. A different color map can be chosen for each (gray, autumn, winter, parula, lipari). After the fit, the T_1 scale runs from 0 to twice the median T_1, and the sliders beside the maps step through their slices and dynamics.

The color bars stand beside the T_1 and R^2 maps, with their end values above and below them: the scale's max and min for T_1, 1 and the R^2 cut-off for R^2.

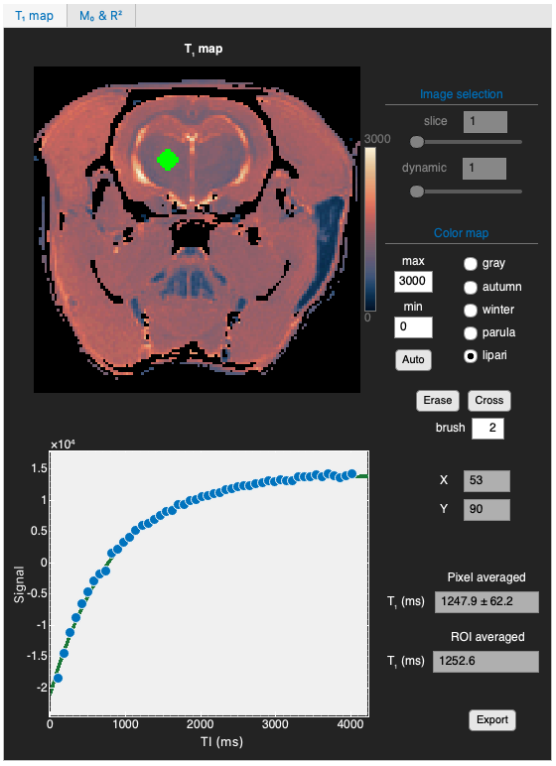
T_1 map **M_0 & R^2**



Step 7: T_1 values of pixels and ROIs

When you click on the T_1 map a cross-hair is shown, with the signal of that pixel as a function of TI (symbols), the fitted curve (line) and its T_1 value (**pixel value**). The signal of an inversion recovery is shown with the sign the fit restored. A red symbol indicates that this point was left out of the fit, according to the list in the Fit parameters tab (see step 5).

You can toggle between the cross-hair and region-of-interest (ROI) values with the **ROI** / **Cross** button. A single ROI can be 'painted' on the T_1 map, in one or more slices, by mouse clicks on the image, with the brush size in pixels set below the button.



For the ROI, two T_1 values are calculated:

- **Pixel averaged** — the average of the T_1 values of the pixels, with their spread: T_1(ROI) = 1 / mean(R_1 of the pixels).
- **ROI averaged** — the T_1 equation is fitted to the average signal intensities in the ROI, producing a single T_1 value for that ROI.

The ROI can be removed with **Erase**. The ROI-averaged signal is written to a CSV file with **Export**, `roiData-<tag>.csv`, beside the inversion times, TRs or flip angles, in a folder you choose.

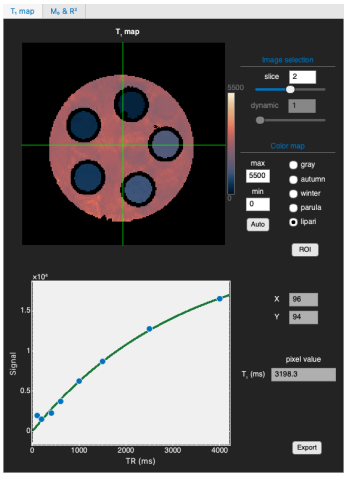
Step 8: Saturation recovery and variable flip angle

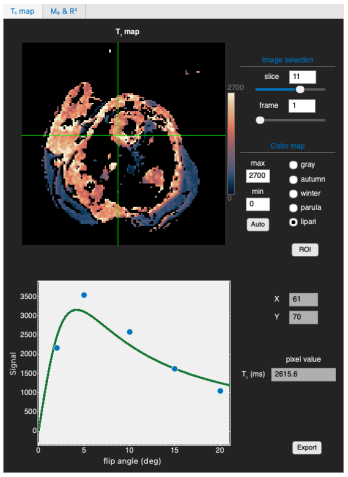
The app also analyzes saturation recovery data. The data need to contain an array of TR values. T_1 is obtained by fitting the signal intensity as a function of TR to

> $Signal = A\ (1 - exp(-TR / T_1))$

Variable flip angle series (spoiled gradient echo) are fitted per pixel with DESPOT1. For these series and for saturation recovery, the sliders and the selection lists are named after the TRs or the flip angles; flip angle series are fitted per cardiac frame.

Saturation recovery (phantom) **Variable flip angle (mouse heart)**

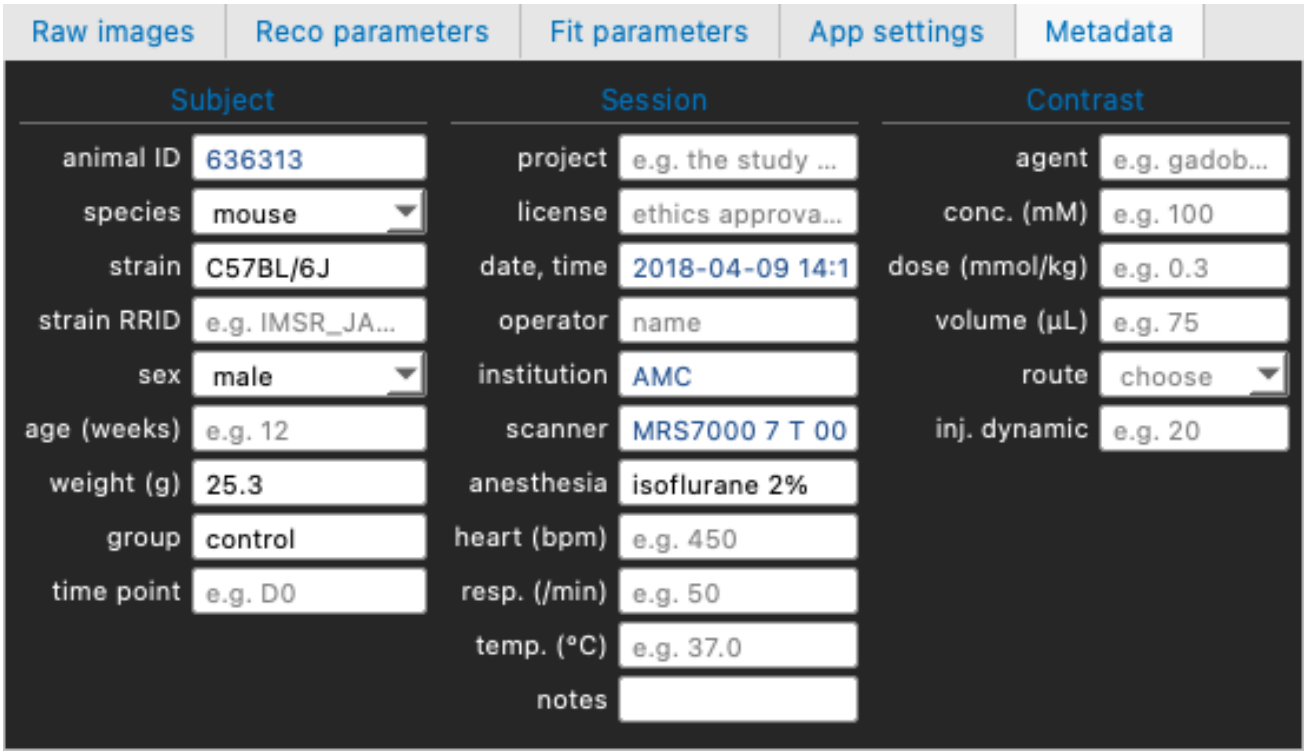




Step 9: Metadata

The **Metadata** tab, beside the Raw images, Reco parameters, Fit parameters and App settings tabs, holds the record of the scan in three columns side by side, which every export of the maps writes with them:

- **Subject:** animal ID, species, strain and its RRID, sex, age, weight, group and time point;
- **Session:** project, ethics license, date and time, operator, institution, scanner, anesthesia, heart and respiration rate, and temperature, with a field for notes;
- **Contrast:** agent, concentration, dose, volume, route and the dynamic it was injected at.



At a load the app fills in what the scan's files say: the animal ID and the time point (from the patient name typed at the console: `20240212_926752_D0` is animal 926752, day 0), the date and time, the institution and the scanner, and the project. What the console leaves at its standard values (sex O, weight 0, the login as the operator) is left empty.

What was filled in for an earlier scan of the same exam is filled in again: the animal's details are typed only once per exam. The date and time, the heart and respiration rate, the temperature, the notes and the injection's dynamic belong to one scan and are never taken from another.

Your institution, operator, anesthesia, species, strain and strain RRID are kept for later exams: they are shown, in blue, from the start, and a scan's own value (from its files or its exam) takes their place where it has one.

A value the app filled in is shown in blue; every field can be corrected, and then counts as yours. Species, sex and route are lists to choose from, or to type in; numbers are kept with their units (a number field holding something else is edged in red). The species and sex are written with their codes (NCBI Taxonomy, PATO), so the record can be read by other software as well as by people.

Step 10: Exporting the maps

There are two ways to export the T_1, M_0 and R^2 maps.

1. **DCM / NIFTI** — exports the maps in DICOM and NIfTI format for further processing in third-party software, and the T_1 map also as a BIDS quantitative map (`BIDS/derivatives/t1mapp/sub-<tag>/anat/ sub-<tag>_T1map.nii.gz`, in seconds; the DICOM and NIfTI maps are in ms). The DICOM maps carry the species, strain, sex, age and weight of the animal and the contrast agent, where the Metadata tab has them.
2. **GIF** — exports the T_1, M_0 and R^2 maps as animated GIF files, with two pictures of the app's window.

When the MRD file lies in the scanner's folders, `…/MRD/<study>/<scan>`, the exports go to `…/Data/<study>/DICOM`, `Nifti`, `BIDS` and `GIF`, each into a new numbered folder; otherwise the app asks for a folder. The ROI data of step 7 go to a folder of your choice. The folder dialog opens in the folder chosen last. The folder and file names the app makes from the scan's name hold no spaces: a space, or any character other than a letter, a digit, `-`, `_` or `.`, becomes `_`.

Every export of the maps also writes, in `…/Data/<study>/Metadata/<tag>T1/<n>` (or under the folder asked for):

- `metadata.json` — the Metadata tab's record;
- `t1mapp-report_<tag>.json` and `.txt` — the report of the export: the app and its libraries, the scan and a checksum of each of its files, its acquisition and the series that was fitted, every step of the session with its parameters (the reconstruction, the registration, the mask and your changes to it, the fit with its settings), the statistics of each map per slice over the fitted pixels within the mask (with the number of pixels left out), and every file the export wrote with its checksum;
- `ro-crate-metadata.json` — the same as an RO-Crate (https://www.researchobject.org/ro-crate/), a standard description of the folder for data repositories.

The paths in them are relative to the study folder, so they hold no names of your own folders.

Step 11: Exit

Press **Exit** to shut down the program. Closing the window does the same, also while a step runs: a reconstruction, registration or fit is stopped, the window disappears, and the program closes as soon as the step has ended.

7. Credits

- **BART toolbox** — https://mrirecon.github.io/bart/. M. Uecker et al., Berkeley Advanced Reconstruction Toolbox, Proc. Intl. Soc. Mag. Reson. Med. 23, 2486 (2015).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019).
- **elastix** — S. Klein, M. Staring, K. Murphy, M.A. Viergever, J.P.W. Pluim, elastix: a toolbox for intensity based medical image registration, IEEE Transactions on Medical Imaging 29(1), 196–205 (2010). https://elastix.dev
- **PCA denoising** — J. Veraart, D. S. Novikov, D. Christiaens, B. Ades-Aron, J. Sijbers, E. Fieremans, Denoising of diffusion MRI using random matrix theory, NeuroImage 142, 394–406 (2016).
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **DESPOT1** — S.C.L. Deoni, B.K. Rutt, T.M. Peters, Rapid combined T1 and T2 mapping using gradient recalled acquisition in the steady state, Magn Reson Med 49(3), 515–526 (2003).
- **Color maps** — F. Crameri, Scientific colour maps (lipari), Zenodo, doi:10.5281/zenodo.1243862.
- **NIfTI conversion** — dcm2niix: C. Rorden et al., https://github.com/rordenlab/dcm2niix.